

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Muhammad Sayyid Tsabit - 5024241013

2026

1 Pendahuluan

1.1 Latar Belakang

Sistem embedded dipakai hampir di semua perangkat elektronik, dari peralatan rumah tangga sampai node IoT dan pengendali industri. Inti dari sistem tersebut adalah mikrokontroler, yaitu chip yang menggabungkan CPU, memori, dan periferal sehingga dapat langsung membaca sensor dan menggerakkan aktuator. Sebelum membangun sistem yang lebih besar, praktikan perlu menguasai dasarnya terlebih dahulu: menyiapkan lingkungan pengembangan, memasukkan program ke chip, dan memakai pin I/O untuk berinteraksi dengan dunia luar.

Modul 1 membahas dasar tersebut dengan dua platform, yaitu STM32 Blackpill yang berbasis ARM Cortex-M4 dan ESP32 yang memiliki WiFi dan Bluetooth terintegrasi. Keduanya diprogram menggunakan PlatformIO dengan framework yang berbeda, yaitu Arduino dan ESP-IDF, sehingga praktikan dapat melihat perbedaan antara kemudahan penulisan program dan kendali terhadap perangkat keras. Selanjutnya pin GPIO dipakai dalam tiga mode: keluaran digital, masukan digital, dan masukan analog. Pada pembacaan tombol, praktikan menemui dua masalah yang selalu muncul pada rangkaian nyata, yaitu pin yang mengambang (floating) dan getaran kontak (bouncing). Jika tidak ditangani, keduanya membuat program membaca masukan yang salah, misalnya pada keypad atau panel kontrol.

1.2 Dasar Teori

1.2.1 PlatformIO dan Visual Studio Code

PlatformIO adalah lingkungan pengembangan untuk sistem *embedded* yang bisa dipakai pada banyak *board* dan *framework*, seperti Arduino, STM32Cube, dan ESP-IDF (PlatformIO, n.d.). Pada chip yang sama, *framework* dapat diganti cukup dengan mengubah file `platformio.ini`. File ini berisi pengaturan *board*, *framework*, dan protokol *upload*. Selain itu, sebuah *project* PlatformIO memiliki folder `src` untuk program utama, `include` untuk file *header*, dan `lib` untuk *library* tambahan.

1.2.2 STM32 Blackpill dan Framework Arduino

Blackpill adalah *board* dengan mikrokontroler STM32F401CC yang memakai inti ARM Cortex-M4 32-bit. *Clock*-nya mencapai 84 MHz, dengan memori *Flash* 256 KB dan RAM 64 KB. LED bawaannya berada di pin PC13 dan menyala ketika pin diberi logika LOW, sedangkan tombol pengguna berada di pin PA0. *Board* ini tidak memiliki *programmer* bawaan, sehingga program dimasukkan lewat ST-Link eksternal menggunakan protokol SWD. Protokol ini memakai jalur SWDIO untuk data dan SWCLK untuk *clock*, ditambah sambungan GND dan 3.3V (STMicroelectronics, n.d.).

STM32 dapat diprogram dengan *framework* Arduino (STM32duino) yang menyediakan fungsi seperti `pinMode()`, `digitalWrite()`, dan `delay()`. Pin juga bisa ditulis langsung dengan label portnya, misalnya PC13. *Framework* ini berjalan di atas HAL milik STMicroelectronics, sehingga pengaturan register GPIO yang biasanya dilakukan sendiri saat memakai HAL sudah ditangani oleh `pinMode()` (STM32duino, n.d.).

1.2.3 ESP32 dan Framework ESP-IDF

ESP32 adalah mikrokontroler 32-bit *dual-core* (Xtensa LX6) yang sudah dilengkapi WiFi dan Bluetooth. *Board* DevKit-nya memiliki konverter USB-to-Serial, sehingga dapat diprogram langsung lewat kabel USB tanpa *programmer* tambahan.

ESP-IDF adalah *framework* resmi dari Espressif yang berjalan di atas FreeRTOS (Espressif Systems, n.d.-a). Program tidak dimulai dari `setup()` dan `loop()`, tetapi dari fungsi `app_main()` yang berjalan sebagai sebuah *task*. Untuk memberi jeda, ESP-IDF memakai `vTaskDelay()`. Fungsi ini membuat *task* menunggu selama beberapa *tick*, dan selama itu *scheduler* dapat menjalankan *task* lain. Fungsi `pdMS_TO_TICKS()` dipakai untuk mengubah satuan milidetik menjadi *tick*. Pada *framework* Arduino, hal-hal tersebut tersembunyi sehingga lebih mudah dipakai, tetapi akses langsung ke *task*, *semaphore*, dan *queue* menjadi terbatas. Karena itu, ESP-IDF lebih cocok untuk pengembangan produk IoT yang lebih kompleks.

1.2.4 Pull-up dan Pull-down Resistor

Pin GPIO yang dipakai sebagai *input* memiliki impedansi yang sangat tinggi, sehingga arus yang masuk ke pin hampir nol. Jika tombol tidak ditekan dan pin tidak tersambung ke VCC maupun GND, tegangan pin akan terpengaruh *noise* di sekitarnya. Kondisi ini disebut *floating*, dan pin dapat terbaca HIGH atau LOW secara acak.

Agar kondisi awal pin pasti, digunakan resistor *pull-up* atau *pull-down*. Pada *pull-up*, pin disambungkan ke 3.3V lewat resistor, sehingga terbaca HIGH saat tombol tidak ditekan dan LOW saat tombol ditekan karena pin tersambung ke GND. Pada *pull-down*, pin disambungkan ke GND lewat resistor, sehingga terbaca LOW saat tombol tidak ditekan dan HIGH saat tombol ditekan karena pin tersambung ke 3.3V. Resistor $10\ \mathrm{k\Omega}$ umum dipakai karena nilainya cukup kecil untuk menahan level pin, tetapi cukup besar sehingga arus saat tombol ditekan hanya sekitar $3.3\ \mathrm{V}/10\ \mathrm{k\Omega} = 0.33\ \mathrm{mA}$.

Resistor ini bisa dipasang di luar mikrokontroler (eksternal) atau diaktifkan lewat program (internal). Pada ESP32 dengan *framework* Arduino, *pull-up* internal diaktifkan dengan `pinMode(pin, INPUT_PULLUP)` dan *pull-down* internal dengan `pinMode(pin, INPUT_PULLDOWN)`. Pin *input-only* (GPIO 34 sampai 39) tidak memiliki resistor *pull* internal, sehingga jika dipakai untuk tombol harus memakai resistor eksternal (Espressif Systems, n.d.-b).

1.2.5 Debouncing

Kontak pada tombol mekanik tidak langsung menyambung atau terputus dengan bersih. Saat tombol ditekan atau dilepas, kontaknya memantul beberapa kali sehingga sinyal berganti antara HIGH dan LOW selama beberapa milidetik hingga puluhan milidetik. Mikrokontroler membaca pin jauh lebih cepat dari itu, sehingga setiap pantulan terbaca sebagai penekanan tersendiri. Akibatnya, satu kali tekan bisa tercatat beberapa kali.

Debouncing dilakukan agar perubahan status tombol hanya diterima jika sinyalnya sudah stabil selama waktu tertentu. Pada cara *software*, waktu setiap perubahan sinyal dicatat dengan `millis()`. Status baru dianggap sah jika selisih waktunya sudah melewati `DEBOUNCE_DELAY`, misalnya 20 sampai 50 ms. Cara ini tidak memakai `delay()`, sehingga program tetap berjalan selama menunggu. Nilai *delay* perlu dipilih dengan seimbang. Jika terlalu kecil, pantulan masih bisa lolos, dan jika terlalu besar, tombol terasa lambat dan penekanan yang cepat bisa terlewat. Selain lewat program, *debouncing*

juga bisa dilakukan dengan perangkat keras, misalnya filter RC atau *Schmitt trigger*.

1.2.6 ADC sebagai Mode Akses GPIO Analog

Input digital hanya mengenal dua kondisi, yaitu HIGH dan LOW, padahal banyak besaran fisik seperti cahaya dan suhu berubah secara kontinu. Untuk membacanya digunakan ADC (*Analog-to-Digital Converter*), yaitu komponen yang mengubah tegangan analog menjadi angka digital. ADC dengan resolusi n bit memiliki 2^n tingkat nilai.

ADC pada ESP32 secara *default* memiliki resolusi 12 bit, sehingga menghasilkan nilai 0 sampai 4095 untuk tegangan sekitar 0 sampai 3.3V. Satu tingkat nilai setara dengan kurang lebih $3.3/4095 \approx 0.8\ \mathrm{mV}$, dan tegangan dapat dihitung dari nilai mentahnya dengan $V = \text{nilai} \times 3.3/4095$. Pembacaan dilakukan dengan `analogRead(pin)` pada *framework* Arduino. Hasil bacanya tidak sepenuhnya linear, terutama di dekat batas rentang. ADC2 juga tidak dapat dipakai penuh saat WiFi aktif, sehingga ADC1 (misalnya GPIO 32 sampai 39) lebih disarankan (Espressif Systems, n.d.-b).

Pada praktikum ini, ADC dipakai untuk membaca LDR, yaitu resistor yang nilainya turun ketika cahaya makin terang. LDR dirangkai sebagai pembagi tegangan dengan resistor $10\ \mathrm{k\Omega}$, dengan LDR di sisi 3.3V dan resistor di sisi GND. Tegangan keluarannya dinyatakan sebagai

$$V_o = V_{in} \times \frac{R_1}{R_{LDR} + R_1} \tag{1}$$

Saat gelap, resistansi LDR besar sehingga V_o kecil. Saat terang, resistansi LDR turun sehingga V_o naik. Dengan begitu, perubahan cahaya diubah menjadi perubahan tegangan yang bisa dibaca ADC.

2 Tugas Pendahuluan

1. Framework Arduino menyederhanakan program menjadi setup() dan loop() dengan fungsi siap pakai seperti pinMode() dan digitalWrite(). Kelebihannya mudah dipelajari, kodenya singkat, dan banyak library, tetapi akses ke fitur tingkat rendah seperti pengaturan task terbatas. ESP-IDF adalah framework resmi Espressif yang berjalan di atas FreeRTOS, dengan program yang dimulai dari app_main() sebagai task (Espressif Systems, n.d.-a). Kelebihannya kendali atas fitur ESP32 lebih luas dan cocok untuk produk IoT, tetapi lebih sulit dipelajari dan kodenya lebih panjang.

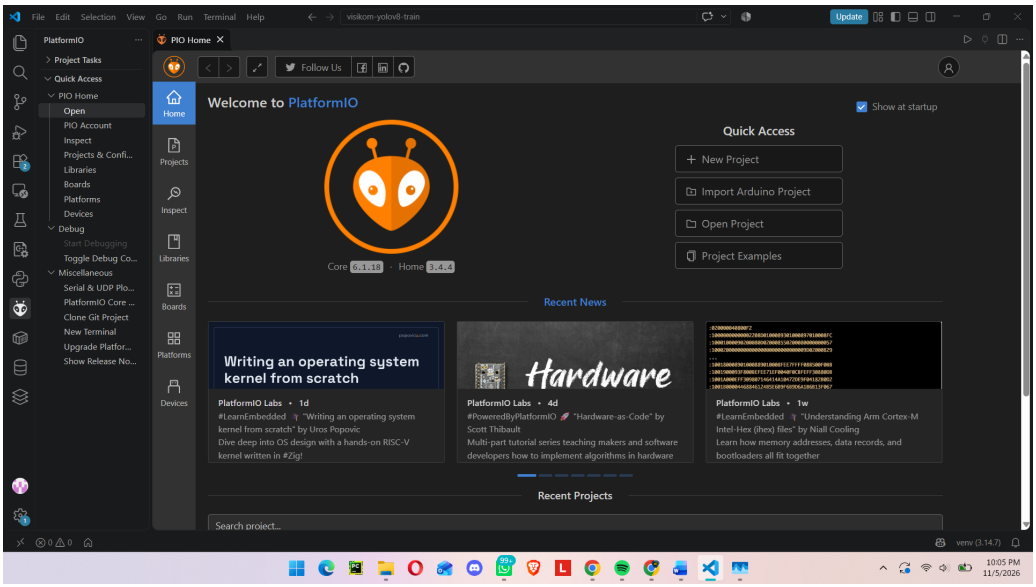
2. Blackpill hanya memuat mikrokontroler STM32F401CC dan tidak memiliki chip yang menghubungkan USB laptop ke jalur pemrogramannya. Mikrokontroler ini diprogram lewat antarmuka SWD, sehingga dibutuhkan ST-Link yang mengubah data USB menjadi sinyal SWD (STMicroelectronics, n.d.). ESP32 DevKit sudah memiliki chip USB-to-Serial (CP2102 atau CH340) yang mengubah data USB menjadi data serial UART. Karena ESP32 punya bootloader yang menerima program lewat UART, program bisa dikirim langsung lewat kabel USB.

3. Pada pull-up, pin dihubungkan ke 3.3V lewat resistor sehingga terbaca HIGH saat tombol tidak ditekan, dan LOW saat tombol ditekan karena pin tersambung ke GND. Pada pull-down, pin dihubungkan ke GND lewat resistor sehingga terbaca LOW saat tombol tidak ditekan, dan HIGH saat tombol ditekan karena pin tersambung ke 3.3V. Jadi logika pull-down berkebalikan dengan pull-up.

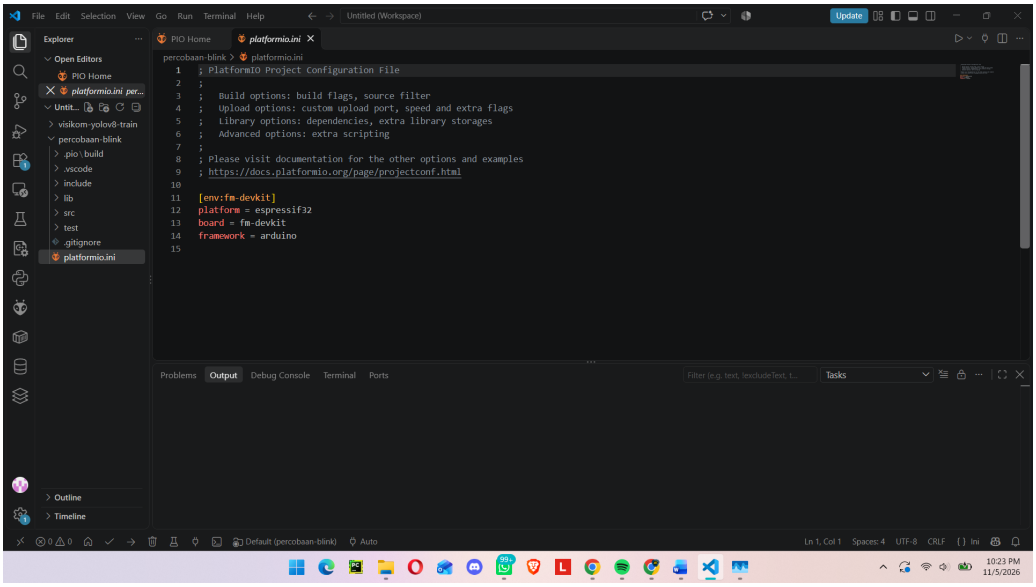
4. Bouncing adalah perubahan sinyal HIGH dan LOW yang berulang dalam waktu singkat saat tombol ditekan atau dilepas. Hal ini terjadi karena kontak logam di dalam tombol memantul sebelum benar-benar menyambung atau terlepas, sehingga satu kali tekan bisa terbaca beberapa kali. Cara mengatasinya antara lain debouncing lewat software, yaitu menerima perubahan status hanya jika sinyal stabil selama waktu tertentu (misalnya 20 sampai 50 ms) menggunakan millis(), dan lewat hardware, yaitu memasang filter RC atau rangkaian Schmitt trigger pada tombol.

5. ADC mengubah tegangan analog menjadi angka digital dengan mengambil sampel tegangan, membandingkannya dengan tegangan referensi untuk dibagi ke dalam sejumlah tingkat nilai, lalu menyatakan tingkat tersebut sebagai bilangan biner. ADC dengan resolusi n bit memiliki 2^n tingkat nilai. Makin besar jumlah bit, makin kecil selisih tegangan yang bisa dibedakan, sehingga hasil bacanya makin presisi. Dengan referensi 3.3V, ADC 8 bit memiliki selisih sekitar 12.9 mV per tingkat, sedangkan ADC 12 bit pada ESP32 sekitar 0.8 mV (Espressif Systems, n.d.-b).

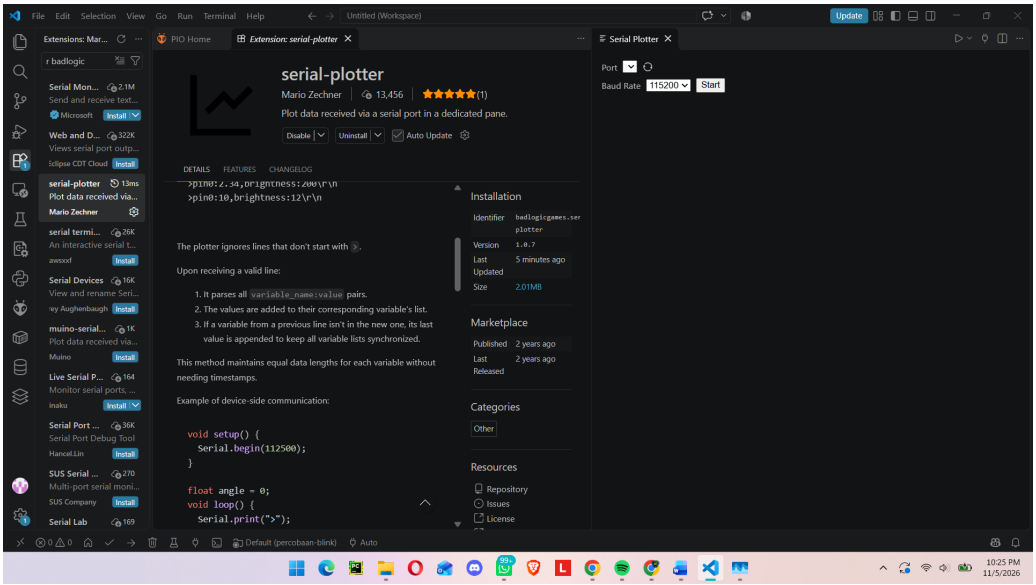
6. Set-up dilakukan dengan memasang Visual Studio Code, lalu ekstensi PlatformIO IDE dan ekstensi Serial Plotter dari menu Extensions. Setelah itu dibuat project untuk ESP32 dengan framework ESP-IDF dan project untuk STM32 Blackpill, lalu dicoba build untuk memastikan toolchain sudah terpasang. Hasilnya ditunjukkan pada Gambar 1 sampai Gambar 3.



Gambar 1: VSCode dengan ekstensi PlatformIO



Gambar 2: Project ESP32 (ESP-IDF) dan STM32 Blackpill berhasil di-build



Gambar 3: Ekstensi Serial Plotter